Transacciones

Aclaración: a diferencia de la separación entre carpetas de procedimientos y transacciones sugerida por la cátedra en el repositorio, en nuestra solución implementamos la lógica transaccional de forma encapsulada y directa dentro del propio procedimiento RegistrarCompraBoleto para garantizar una ejecución atómica.

Al empezar, el procedimiento se fija en la tabla COMPRA_FUNCION,
si la butaca que eligió el usuario ya está ocupada, salta un error con el RAISERROR, cancela todo y hace un ROLLBACK al instante.
Si la butaca está libre, calculamos automáticamente el próximo `id_compra` que corresponde y metemos el registro principal en la tabla `COMPRA`.
Luego guardamos el detalle, enlazamos esa compra con la función y la butaca exacta en la tabla COMPRA_FUNCION.
Y por último, el cierre con red de seguridad. Todo esto está metido dentro de un bloque TRY…CATCH. Si todo salió bien, metemos un COMMIT TRANSACTION y el boleto queda registrado con exito. Pero si llega a fallar cualquier cosa, el sistema salta automáticamente al CATCH y hace un ROLLBACK general para evitar que los datos queden suelto o tablas inconsistentes.

```sql
    -- 3. INICIO DE TRANSACCIÓN (Garantía ACID)
    BEGIN TRANSACTION;

    BEGIN TRY
        -- A. Verificar si la butaca ya fue ocupada para esta función
        IF EXISTS (SELECT 1 FROM COMPRA_FUNCION WHERE id_funcion = @id_funcion AND id_butaca = @id_butaca)
        BEGIN
            RAISERROR('La butaca seleccionada ya no se encuentra disponible.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        -- B. Obtener el siguiente id_compra correlativo para tu esquema actual
        DECLARE @nuevo_id_compra INT;
        SELECT @nuevo_id_compra = ISNULL(MAX(id_compra), 0) + 1 FROM COMPRA;

        -- C. Registrar la cabecera de la compra
        INSERT INTO COMPRA (id_compra, fecha, dni, id_beneficio)
        VALUES (@nuevo_id_compra, CAST(GETDATE() AS DATE), @dni, @id_beneficio);

        -- D. Registrar el detalle en COMPRA_FUNCION
        INSERT INTO COMPRA_FUNCION (cantidad, id_compra, id_funcion, id_butaca)
        VALUES (1, @nuevo_id_compra, @id_funcion, @id_butaca);

        -- Confirmar cambios atómicamente
        COMMIT TRANSACTION;
        PRINT '¡Boleto registrado y butaca asignada con éxito!';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END
```

Evidencia del funcionamiento de transacciones en nuestro sistema:

```sql
105    PRINT '--- EJECUTANDO CASO 2: Butaca ya ocupada ---';
106    EXEC dbo.sp_RegistrarCompraBoleto
107        @dni = 31222333,
108        @id_funcion = 1,
109        @id_butaca = 2,
110        @id_beneficio = NULL;
111    GO
112    -- *Resultado esperado:* Mensaje de error controlado: "La butaca seleccionada ya no se encuentra disponible."
113
114
115    -- ================================================================================
116    -- CASO DE PRUEBA 3: Regla de Negocio - Función Iniciada (> 15 minutos)
```

20 % | No se encontraron problemas. | Línea: 109, Carácter: 30 | SPC

Mensajes

```
--- EJECUTANDO CASO 2: Butaca ya ocupada ---
Mensaje 50000, nivel 16, estado 1, procedimiento dbo.sp_RegistrarCompraBoleto, línea 68 [línea de inicio de lote 104]
La butaca seleccionada ya no se encuentra disponible.
```